KinovaSDK-ROS2: A Python SDK for the Kinova Gen3 Lite Robot

Alexander Cerón-Correa*
*ACCEDER, Universidad Militar Nueva Granada, Bogotá, Colombia
{alexander.ceron@unimilitar.edu.co}

***Abstract*—Programming robotic manipulators through Robot Operating System (ROS 2) often requires users to interact directly with low-level communication mechanisms, including topics, actions, services, and controller interfaces. Although these components provide flexibility, they also increase the complexity of application development, particularly for educational, research, and rapid prototyping scenarios. This work presents a modular Python library that provides a high-level application programming interface (API) for controlling the Kinova Gen3 Lite robotic manipulator. The proposed architecture is based on a central KinovaRobot class that integrates independent software modules using a mixin-based design. By encapsulating the underlying ROS 2 communication mechanisms, the proposed API simplifies robot programming while maintaining extensibility and modularity. The resulting software architecture facilitates code reuse, improves maintainability, and provides a foundation for developing more advanced robotic applications in research and education.**

***Index Terms*—Robotics, ROS 2, Manipulators, Software Architecture, Kinova Gen3 Lite, SDK**

I. Introduction

Robotic manipulators are designed to perform physical tasks—such as lifting, moving, and placing objects—in a variety of environments. They are therefore important systems in robotics [1]. A typical manipulator consists of links connected by joints, forming an open kinematic chain that terminates in an end effector. The end-effector may be a tool, a gripper, a robotic hand, or a suction cup, among other possibilities [2].

A key feature of robotic manipulators is their ability to interact directly with the environment, making them useful in both industrial and non-industrial applications. Industrial manipulators perform many essential manufacturing tasks with precision [1].

Key applications include material handling (pick-and-place), palletizing and packaging, welding and assembly, machine tending, and quality control. Outside industry, they are used in medicine (e.g., assisted surgery), aerospace (e.g., satellite maintenance), agriculture, logistics, nuclear environments, and domestic settings [3].

Collaborative robotic arms (cobots) are designed to work alongside humans in a range of sectors. Their applications include material handling (pick-and-place), CNC machine tending, welding, screwdriving, and palletizing. They can support productivity and flexible production without requiring conventional safety cages [4].

The Fourth Industrial Revolution, or Industry 4.0, is characterized by digital transformation and the integration of the physical, digital, and biological worlds, changing how we live, work, and produce goods and services [5]. Driven by new production requirements—such as the need for high flexibility in product development—it has become necessary to develop increasingly accessible programming tools or software for robotics.

In recent years, various software tools for robotics have been developed. Notable among these is an open-source framework—or middleware—that facilitates robot software development. Known as ROS (Robot Operating System), it is installed on top of an existing operating system (typically Linux); therefore, it is not an operating system in the traditional sense (like Windows or Linux) [6].

ROS 2 was developed in response to the increasing complexity of robotics applications and the need for an environment that reduces the risk of failure. It has been adopted in recent applications for deployment across various environments [7].

Given the complexity of modern robotic systems, controlling them using current tools—such as ROS and ROS2, which are becoming industry standards—involves considerations that can make the learning curve quite steep. This motivates the development of tools that streamline application development.

Among commercially available robotic manipulators, the Kinova Gen3 Lite is used in education and research [8]. It features six degrees of freedom and is equipped with a gripper-style end-effector. Several software tools support this robot, including MATLAB, Python, and ROS 2, which is used in interactive applications [9]. However, utilizing the robot presents challenges, as programming it using ROS 2 topics, actions, and services requires advanced knowledge. Consequently, operation sometimes necessitates a large number of instructions, resulting in limited flexibility for practical application development—a situation that has led to:

- The difficulty the official API poses for students and researchers.
- The need for a simpler programming interface.

For these reasons this paper presents KinovaSDK-ROS2, a Python SDK that abstracts the complexity of ROS 2 communication and provides an intuitive programming interface for the Kinova Gen3 Lite robot. This robot is shown in Figure 1.

The main contributions of this paper are:

A modular Python SDK for the Kinova Gen3 Lite. A mixin-based software architecture. A layered integration between

ROS2 and the official Kinova SDK. A reusable interface intended for education and research.



Fig. 1: Kinova Gen3 Lite Robot.

II. Software Architecture

Software development for robotic systems has evolved over recent decades alongside advances in robotics, becoming an important area of study [10], [11]. These advances have led to increasingly complex software, making software engineering practices essential. Such practices include layered architectures, class specifications, libraries, and execution-flow design [12].

The proposed software architecture is designed to provide a high-level Python API for controlling the Kinova Gen3 Lite manipulator while preserving compatibility with the official Kinova SDK. The design separates user-facing functionality, ROS 2 communication, and vendor-specific capabilities into independent layers.

The main design goals are:

- Provide an intuitive Python interface.
- Reuse ROS 2 infrastructure for motion execution.
- Reuse the official Kinova inverse kinematics solver instead of implementing a custom solver.
- Isolate vendor-specific code inside a dedicated bridge package.

The design is based on a modular architecture using *Mixins*, where each module is responsible for implementing a specific robot capability:

- Joint state management.
- Motion control.
- Gripper control.
- Pose management.
- Sequence execution.
- Inverse kinematics.

A. Layered architecture

The proposed framework follows a layered software architecture that separates the user application, the high-level programming interface, the ROS 2 communication infrastructure, and the manufacturer-specific software. This organization improves modularity, supports future extensions, and allows new functionality to be integrated without changing the high-level user API.

Unlike the official Kinova ROS2 driver, which mainly exposes motion controllers and robot state interfaces, the proposed framework introduces an additional communication layer that exposes advanced SDK functionalities through ROS2 services. As a consequence, users can access both standard robot operations and proprietary SDK capabilities using a unified Python interface.

Figure 2 illustrates the four software layers composing the framework.

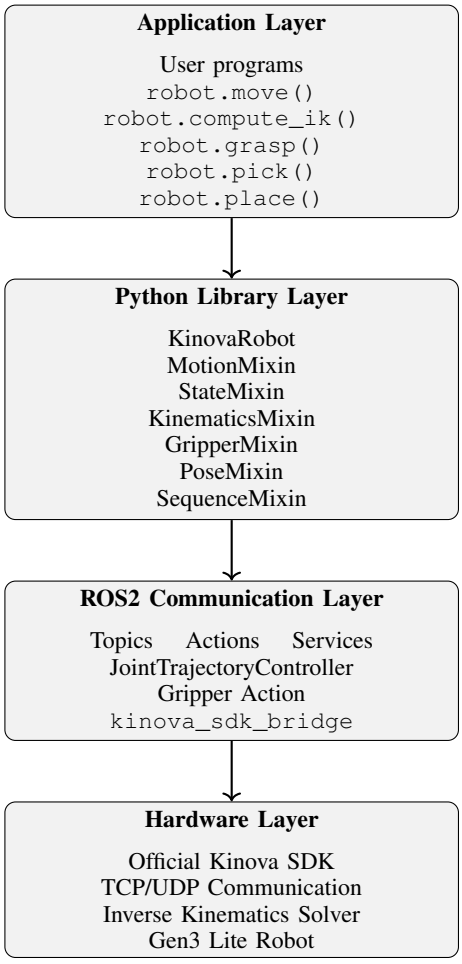


Fig. 2: Layered software architecture of the proposed Kinova framework.

The architecture is organized into four abstraction layers.

1) Application Layer: The Application Layer corresponds to the user-facing interface of the framework. Robot programs are written using intuitive object-oriented

commands such as `robot.move()`, `robot.home()`, `robot.compute_ik()`, and `robot.grasp()`. No knowledge of ROS2 communication mechanisms or the proprietary Kinova SDK is required to develop robot sequences.

Typical application code consists of intuitive instructions such as

```
robot.home()
robot.move(...)
robot.compute_ik(...)
robot.zero()
robot.move_joint(...)
robot.move_cartesian(...)
```

This abstraction significantly reduces the programming complexity compared to directly using ROS2 actions, services, or the manufacturer's SDK.

2) Python Library Layer: The Python Library Layer implements the object-oriented API exposed to the user. The central component is the `KinovaRobot` main class, which aggregates multiple Mixins responsible for different robot capabilities.

Each Mixin encapsulates a specific functionality:

- MotionMixin: joint-space and Cartesian motion commands.
- StateMixin: robot state monitoring.
- GripperMixin: gripper control.
- PoseMixin: persistent storage and retrieval of named poses.
- SequenceMixin: execution of complex task sequences.
- KinematicsMixin: inverse kinematics computation.

This modular organization simplifies future extensions since new functionalities can be incorporated by adding new Mixins without modifying the remaining components.

3) ROS2 Communication Layer: The communication layer is responsible for interfacing the Python API with the robot through ROS2.

Most robot operations are implemented using the standard ROS2 communication mechanisms already provided by the official Kinova ROS2 driver, including:

- Joint trajectory actions.
- Gripper actions.
- Robot state topics.
- TF transformations.
- Controller Manager services.

However, the official driver does not expose the manufacturer's inverse kinematics solver through ROS2.

To overcome this limitation, this work introduces a new ROS2 package named `kinova_sdk_bridge`. The proposed architecture incorporates a ROS2 bridge that exposes the official Kinova inverse kinematics functionality as a service.

The Python library therefore invokes the bridge service transparently, while retaining the same high-level programming interface.

4) Hardware Layer: The Hardware Layer corresponds to the proprietary Kinova SDK and the physical Gen3 Lite manipulator.

The SDK establishes TCP and UDP communication sessions with the robot controller and provides access to low-level functionalities, including:

- Motion execution.
- Robot feedback.
- Gripper control.
- Inverse kinematics solver.
- Robot configuration.

The proposed framework does not reimplement inverse kinematics. Instead, it invokes the implementation provided by the official SDK, preserving consistency with the manufacturer's software. Compatibility with future firmware or SDK updates depends on the continued support of the relevant API.

B. Class architecture

1) KinovaRobot: The `KinovaRobot` class inherits from `rclpy.node.Node` and acts as the central point of the library.

Its responsibilities are:

- Creating the ROS 2 node.
- Managing communication with the controllers.
- Integrating the different modules.

2) MotionMixin: The module implements the transmission of joint trajectories.

The motion flow is:

$$q_d \rightarrow JointTrajectory \rightarrow Controller \rightarrow Robot$$

3) StateMixin: This module retrieves the robot's current configuration from:

`/joint_states`

Joint positions are represented as:

$$q = [q_1, q_2, q_3, q_4, q_5, q_6]$$

4) KinematicsMixin: This module integrates inverse kinematics into the Python interface. The architecture used is:

The external solver calculates:

$$f^{-1}(x, y, z, R_x, R_y, R_z) = (q_1, q_2, q_3, q_4, q_5, q_6)$$

Subsequently, these values are sent to the `MotionMixin` module to execute the movement.

C. Execution Flow of the Proposed Architecture

One of the main contributions of the proposed framework is the transparent integration between a high-level Python API and the official Kinova SDK while preserving the standard ROS2 communication mechanisms.

From the user's perspective, robot programming consists of invoking intuitive object-oriented methods such as `robot.move()`, `robot.home()`, or

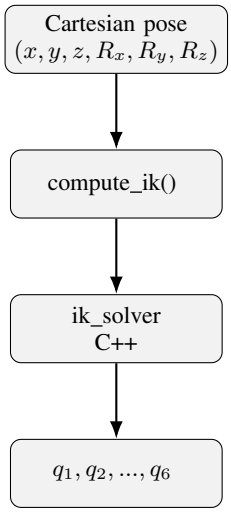


Fig. 3: Inverse kinematics calculation workflow.

`robot.compute_ik()`. Internally, these commands traverse several abstraction layers before reaching the physical manipulator.

Figure 4 illustrates the complete execution path of an inverse kinematics request.

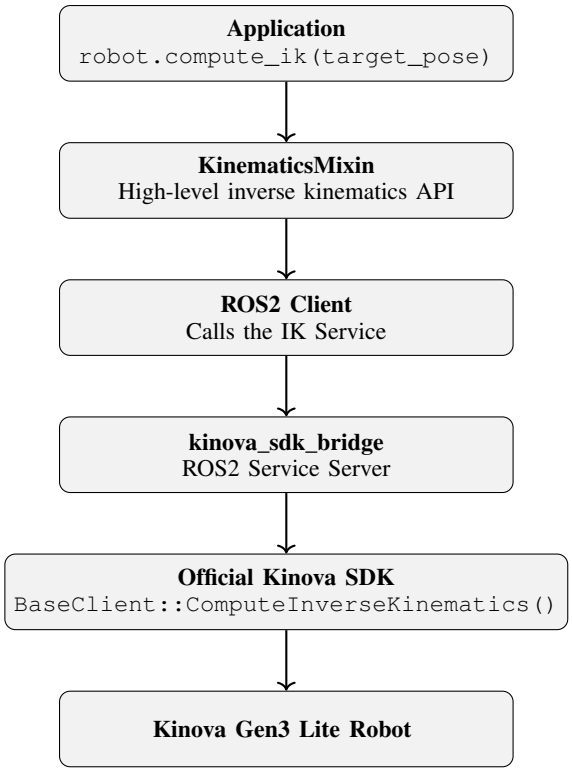


Fig. 4: Execution flow of an inverse kinematics request in the proposed framework.

The execution begins in the user application, where the programmer invokes the method

`robot.compute_ik(target_pose)`

without requiring any knowledge of ROS2 messages or the proprietary Kinova SDK.

The request is received by the `KinematicsMixin`, which provides the high-level inverse kinematics interface of the framework. This module converts the desired Cartesian pose into a ROS2 service request.

The request is transmitted to the `kinova_sdk_bridge` package through the ROS2 middleware. Unlike the original Kinova ROS2 driver, which does not expose inverse kinematics as a ROS2 interface, the proposed bridge implements a dedicated service that internally creates and manages a `BaseClient` object from the official Kinova SDK.

Once the service receives the request, it invokes the manufacturer implementation

`BaseClient::ComputeInverseKinematics()`

which invokes the inverse kinematics solver provided by Kinova.

D. Advantages

Finally, the computed joint configuration is returned through the same communication chain until it reaches the Python application as a standard Python object. Consequently, the entire communication process remains transparent to the user while preserving compatibility with the manufacturer's firmware and avoiding reimplementation of the inverse kinematics algorithm. In summary, the developed SDK offers the following advantages:

- Modular design.
- Separation of concerns.
- Direct reuse of the manufacturer's validated solver.
- No modification of the official ROS 2 driver.
- Future SDK capabilities can be exposed as additional ROS 2 services.

III. Usage Examples

The `kinova_sdk` library provides a Python interface for interacting with the Kinova robotic manipulator through ROS 2. Its API encapsulates common robot operations, including joint-space motion, predefined configurations, inverse kinematics, and Cartesian-space motion. The following examples illustrate these capabilities and demonstrate how the library can be integrated into Python applications.

A. Joint-Space Control

The first example illustrates how to retrieve the current joint positions and command the six robot joints to zero degrees. The `wait_for_robot()` method ensures that the robot is available before commands are issued. The joint positions are reported in degrees, facilitating the interpretation of the robot's configuration. The demonstration of this example can be found in https://youtu.be/KR4ZJhjt98g.

Listing 1: Joint-space control using the `kinova_sdk` API.

```
import rclpy
from kinova_sdk.robot import KinovaRobot
def main():
```

```
    rclpy.init()
    robot = KinovaRobot()
    try:
        robot.wait_for_robot()
        print(robot.get_joint_positions_deg())
        robot.print_joint_positions()
        for joint in range(6):
            robot.move_joint(joint, 0)
    finally:
        robot.destroy_node()
        rclpy.shutdown()
if __name__ == "__main__":
    main()
```

This example demonstrates the abstraction of individual joint commands through a common interface. The target configuration corresponds to setting each joint angle to zero degrees; the resulting end-effector pose depends on the robot's kinematic configuration and joint conventions.

B. Predefined Robot Configurations

The library also provides methods for commanding predefined robot configurations. The following example invokes the `zero()`, `home()`, and `safe()` methods sequentially, with delays between commands to facilitate observation of the transitions.

Listing 2: Sequential execution of predefined configurations.

```
import time
import rclpy
from kinova_sdk.robot import KinovaRobot
def main():
    rclpy.init()
    robot = KinovaRobot()
    try:
        robot.wait_for_robot()
        robot.zero()
        time.sleep(3)
        robot.home()
        time.sleep(5)
        robot.safe()
        time.sleep(5)
        robot.zero()
        time.sleep(3)
        robot.safe()
        robot.home()
    finally:
        robot.destroy_node()
        rclpy.shutdown()

if __name__ == "__main__":
    main()
```

These methods provide a higher-level interface for common robot configurations, avoiding the need for the application developer to specify each joint angle individually. The delays are included for demonstration purposes and do not, by themselves, guarantee that a motion has completed. The definitions of the predefined configurations depend on the library implementation.

C. Inverse Kinematics

The `compute_ik()` method calculates a joint-space solution for a desired end-effector pose. In this example, the target position is specified in Cartesian coordinates, while the orientation is defined by three angular parameters.

Listing 3: Inverse kinematics for a target end-effector pose.

```
import rclpy
from kinova_sdk.robot import KinovaRobot
def main():
    rclpy.init()
    robot = KinovaRobot()
    try:
        robot.wait_for_robot()
        q = robot.compute_ik(
            x=0.40, y=0.10, z=0.20,
            theta_x=180, theta_y=0, theta_z=90
        )
        if q is None:
            print("No␣inverse␣kinematics␣solution
                ␣found")
        else:
            for i, angle in enumerate(q, start=1)
                :
                print(f"Joint␣{i}:␣{angle:.3f}␣deg
                    ")
    except Exception as e:
        print(e)
    finally:
        robot.destroy_node()
        rclpy.shutdown()
if __name__ == "__main__":
    main()
```

The method separates the specification of the desired end-effector pose from the calculation of the corresponding joint configuration. When a solution is returned, the joint angles can be inspected or used by a subsequent motion command. The existence of a returned solution does not, by itself, guarantee that the target is reachable under all operational constraints or that the robot has executed the motion.

D. Cartesian-Space Motion

Finally, the library provides the `move_cartesian()` method to command a desired end-effector pose directly. Unlike the previous example, in which the joint-space solution is explicitly retrieved, this interface encapsulates the motion request at the Cartesian level.

Listing 4: Cartesian motion with a specified target pose and duration.

```
import rclpy
from kinova_sdk.robot import KinovaRobot
def main():
    rclpy.init()
    robot = KinovaRobot()
    try:
        robot.wait_for_robot()
        robot.move_cartesian(
            x=0.40, y=0.25, z=0.10,
            theta_x=180, theta_y=0, theta_z=90,
            duration=4.0
        )
        print("Motion␣command␣completed.")
    finally:
        robot.destroy_node()
```

```
        rclpy.shutdown()
if __name__ == "__main__":
    main()
```

This interface allows applications to specify the desired end-effector position, orientation, and requested motion duration without explicitly managing the individual joint commands. The feasibility of the target pose and the interpretation of the duration depend on the underlying motion implementation and the robot controller. A demonstration of a sequence of movements can be seen in: https://youtu.be/NCM1bpztS5o.

Overall, these examples illustrate how `kinova_sdk` provides a consistent Python interface across joint-space and Cartesian-space operations, while exposing higher-level functionality for predefined configurations and inverse kinematics. This separation can simplify the development of robotic applications by reducing the amount of application-level code required to interact with the manipulator.

IV. Conclusions

The architecture developed for the `kinova_sdk` library provides a high-level abstraction layer for controlling the Kinova Gen3 Lite manipulator using ROS 2. Through a unified interface based on the `KinovaRobot` class, the user can access functionalities such as joint movement, end-effector control, pose management, sequence execution, and inverse kinematics resolution without directly interacting with the robot's internal communication mechanisms.

Adopting a mixin-based architecture allows for a clear separation of component responsibilities, fostering modularity, code reusability, and project maintainability. Each module encapsulates a specific function, reducing coupling between components and facilitating the addition of new capabilities without altering the library's core structure.

Another key aspect of the proposed architecture is the separation between high-level robot programming and computationally intensive kinematic operations. In the current implementation, inverse kinematics is delegated to a ROS2 bridge developed in C++, which interfaces with the official Kinova SDK.

Python provides a simple, easily extensible programming interface, while the C++ bridge provides access to the manufacturer's implementation. This strategy combines the computational efficiency of compiled code with the flexibility and rapid development capabilities offered by this language.

From the user's perspective, the library significantly simplifies the programming of robotic applications. Complex operations—such as executing trajectories, storing configurations, replaying sequences, or moving the robot to a Cartesian pose can be performed using a small set of high-level methods, thereby hiding the complexity associated with the underlying ROS 2 controllers, actions, topics, and services.

The proposed architecture also serves as a solid foundation for developing more advanced applications. The library's modular nature facilitates the future integration of components for trajectory planning, computer vision-based perception, sensor-guided manipulation, machine learning, reactive control, teleoperation, or integration with cyber-physical systems, all while maintaining a consistent interface for the end user.

Finally, the resulting organization transforms a collection of independent examples into a reusable, extensible, object-oriented software library. This evolution facilitates its use in both educational activities and research projects, providing a platform for developing new tools for the programming, experimentation, and validation of algorithms applied to robotic manipulators.

Acknowledgments

The author would like to thank Universidad Militar Nueva Granada for the financing of the project INV_ING 3975 from which this research was supported.

References

[1] B. Siciliano and O. Khatib, Eds., *Springer Handbook of Robotics*, ser. Springer Handbooks. Cham: Springer International Publishing, 2016. [Online]. Available: https://link.springer.com/10.1007/978-3-319-32552-1

[2] J. J. Craig, *Robótica: John J. Craig*. Pearson Educación, 2006, google-Books-ID: hRzOp_qdxG8C.

[3] G. Singh and V. K. Banga, "Robots and its types for industrial applications," *Materials Today: Proceedings*, vol. 60, pp. 1779–1786, Jan. 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2214785321081682

[4] V. Villani, F. Pini, F. Leali, and C. Secchi, "Survey on human–robot collaboration in industrial settings: Safety, intuitive interfaces and applications," *Mechatronics*, vol. 55, pp. 248–266, Nov. 2018. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0957415818300321

[5] H. Lasi, P. Fettke, H.-G. Kemper, T. Feld, and M. Hoffmann, "Industry 4.0," *Business & Information Systems Engineering*, vol. 6, no. 4, pp. 239–242, Aug. 2014. [Online]. Available: https://doi.org/10.1007/s12599-014-0334-4

[6] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Ng, "ROS: an open-source Robot Operating System," vol. 3, Jan. 2009, journal Abbreviation: ICRA Workshop on Open Source Software Publication Title: ICRA Workshop on Open Source Software. [Online]. Available: https://ai.stanford.edu/~mquigley/papers/icra2009-ros.pdf

[7] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, May 2022. [Online]. Available: https://www.science.org/doi/10.1126/scirobotics.abm6074

[8] H. M. Zohour, B. Belzile, and D. St-Onge, "Kinova Gen3-Lite manipulator inverse kinematics: optimal polynomial solution," Feb. 2021, arXiv:2102.01217 [cs.RO]. [Online]. Available: http://arxiv.org/abs/2102.01217

[9] H. Chen, J. Wang, and M. Q.-H. Meng, "Kinova Gemini: Interactive Robot Grasping with Visual Reasoning and Conversational AI," in *2022 IEEE International Conference on Robotics and Biomimetics (ROBIO)*, Dec. 2022, pp. 129–134. [Online]. Available: https://ieeexplore.ieee.org/document/10011896

[10] A. Ahmad and M. A. Babar, "Software architectures for robotic systems," *Journal of Systems and Software*, vol. 122, no. C, pp. 16–39, Dec. 2016. [Online]. Available: https://doi.org/10.1016/j.jss.2016.08.039

[11] H. Andreasson, G. Grisetti, T. Stoyanov, and A. Pretto, "Software Architectures for Mobile Robots," in *Encyclopedia of Robotics*. Springer, Berlin, Heidelberg, 2023, pp. 1–11. [Online]. Available: https://link.springer.com/rwe/10.1007/978-3-642-41610-1_160-1

[12] D. T. Coleman, I. A. Sucan, S. Chitta, and N. Correll, "Reducing the Barrier to Entry of Complex Robotic Software: a MoveIt! Case Study." Italy, 2014, accepted: 2017-06-16T16:10:40Z. [Online]. Available: https://aisberg.unibg.it/handle/10446/87657?__cf_chl_rt_tk=DlCimOQWwINmg59WoRRRgf3SLSlpjq09cZbXiNSdoak-1784087519-1.0.1.1-i_R74zOUX.dQzBo_X9ekNdLrBm4VAzkc_uElWWV5CEU